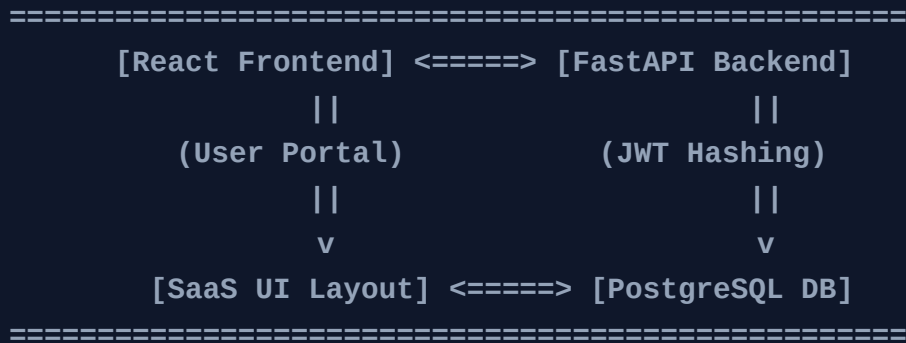


PillSync Platform

Complete Project Implementation & Viva Study Guide



Infosys Springboard Milestone 1 Certification

Written in Simple Semi-Telugu (Telugu & English Transliteration)

Targeting Clean Architecture, RBAC Security, and Seeding

Chapter 1: Project Introduction

1.1 What is PillSync?

PillSync anedi patients, caregivers, and medical admins kosam design chesina ok intelligent medicine reminder platform. Milestone 1 lo memu system basic architecture build chesam, deentlo primary focus user account management and profiles secure cheyyadam midha pettam. Idi regular ga tablets vesukune variki and andulo help chese caregivers ki chala useful.

1.2 Why this project? (Problem Statement)

Mana daily life lo chala mandi medicines time ki vesukotam marchipothuntaru (Forgetfulness). Dheeni valla chronic diseases (like diabetes, BP) badhapade patients ki health chala damage avuthundi. Traditional paper notes or standard alarms clear history ivvalevu. Plus, patients control ela undho relative (caregiver) verify cheyaleka potunnaru. PillSync ee gaps ni fill chesthundhi. Secure role-based database dwara profile details clean ga tracking chesthundhi.

1.3 Objective & Output

Main objective enti ante secure authentication process tho system access control protect cheyyadam. Indhulo 3 major user roles untaru:

- * **Admin:** Full platform monitoring, statistics dashboards and user management capabilities provide chesthundhi.
- * **Patient:** Thana details phone, blood group and emergency contact numbers custom profiles updates chesukuntaru.
- * **Caregiver:** Multiple patients link database settings dynamic status views dashboard roster direct ga monitoring chestharu.

1.4 Technologies Used

Web saas guidelines and instructions follow ayyi ee technology stack utilize chesam:

- * **React.js:** Web frontend responsive client ui layouts write cheydaniki.
- * **FastAPI:** High performance REST APIs endpoints create cheyadaniki.
- * **PostgreSQL:** Relational database tables storage maintain cheydaniki.
- * **JWT & bcrypt:** Secure stateful session management token updates validation kosam.

Chapter 2: Project Architecture

2.1 Multi-Layer SaaS System Flow

Ee project basic ga client-server architecture follow avuthundhi. Client (Frontend React App) direct ga REST APIs call chesi backend endpoint communicate chesthundhi.

```
+-----+
|               REACT FRONTEND LAYER               |
| - Login Form - Admin Dashboard - Profile Manager |
+-----+
|               || (HTTP Axios request with JWT Token)
|               \//
+-----+
|               FASTAPI BACKEND LAYER               |
| - Auth Routers (/login) - Database ORM Session Bindings |
| - Token Signature Verification - Role Checkers         |
+-----+
|               || (SQLAlchemy Sessions)
|               \//
+-----+
|               POSTGRESQL DATABASE                 |
| - Users Table - Profiles Tables - Roles Master      |
+-----+
```

2.2 Modules and Components

Prathi system component clean software development standards match avvaniki custom layer definitions follow chesthundhi:

- * **Frontend Portal:** React router use cheshtu role checking based dynamic redirection logic protect chesthundhi.
- * **FastAPI Middleware:** CORS policy check systems direct request routing headers pass coordinate chesthundhi.
- * **ORM Engine:** SQLAlchemy Base session binding mappings users structure auto load update operations handle chesthundhi.

Chapter 3: Folder Structure

3.1 Complete Project Tree

Memu standard clean code directory layout adopt chesam, files easily accessible and debug-friendly mode config lo untayi:

```
PillSync/
|-- backend/
|   |-- app/
|   |   |-- models/           # SQLAlchemy DB Tables representation
|   |   |   |-- user_models.py
|   |   |-- routers/         # API endpoint routes definitions
|   |   |   |-- admin.py
|   |   |   |-- auth.py
|   |   |   |-- users.py
|   |   |-- schemas/         # Pydantic validation schemas
|   |   |   |-- user_schemas.py
|   |   |-- services/        # Business logic & queries execution
|   |   |   |-- auth_service.py
|   |   |   |-- user_service.py
|   |   |-- utils/           # Hashing and security helpers
|   |   |   |-- security.py
|   |   |-- config.py        # App Environment config settings
|   |   |-- database.py      # Engine creation & session factory
|   |   |-- main.py          # Lifespan startup hooks & CORS settings
|   |-- .env                 # Database connection URL & JWT keys
|-- frontend/
|   |-- src/
|   |   |-- components/      # Reusable layout UI parts
|   |   |   |-- Navbar.jsx
|   |   |   |-- Sidebar.jsx
|   |   |-- pages/           # Viewable screens of application
|   |   |   |-- AdminDashboard.jsx
|   |   |   |-- CaregiverDashboard.jsx
|   |   |   |-- PatientDashboard.jsx
|   |   |   |-- Login.jsx
|   |   |   |-- Register.jsx
|   |   |   |-- Profile.jsx
|   |   |-- App.jsx          # Routing & global auth provider context
|   |-- index.html
|   |-- package.json
```

3.2 Directory Explanations

routers/ files endpoint incoming http request pathways define cheshtayi. services/ lo actual logic and query codes run cheshtunnam. schemas/ request format validations match cheshtundi to avoid garbage inputs.

Chapter 4: Database Schema & PostgreSQL

4.1 What is PostgreSQL?

PostgreSQL is a powerful, open-source object-relational database system (RDBMS). Ee application lo users and profiles security maintain cheyadaniki references maintain chesthundi.

4.2 Database Design & Tables

Milestone 1 design system follows 4 main relational tables schema:

```
-- Roles table mapping user authorizations
CREATE TABLE IF NOT EXISTS roles (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL
);

-- Users base auth credentials
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE RESTRICT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Users table columns properties simple telugu explanation:

- email: dynamic lookup identity. Index index_users_email dynamic filter speed chesthundi.
- role_id: role references mapping, RESTRICT delete configurations user profile integrity protect chesthundi.

```
-- Patient Profile physical fields mapping
CREATE TABLE IF NOT EXISTS patient_profiles (
    id SERIAL PRIMARY KEY,
    user_id INTEGER UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    age INTEGER,
    gender VARCHAR(20),
    blood_group VARCHAR(10),
    address TEXT,
    emergency_contact VARCHAR(100),
    account_status VARCHAR(20) DEFAULT 'Active'
);
```

HELPFUL TIP: ON DELETE CASCADE configure chesthe, user account delete avvagane profile files tables row automate dele
memory system release avuthundi.

Chapter 5: Backend (FastAPI & Python)

5.1 Why FastAPI?

FastAPI is a modern, fast, web framework for building APIs with Python. Deeni dynamic documentation feature (/docs screen) standard REST validations checking chala dynamic speeds deliver chesthundi.

5.2 Virtual Environment & Package Setup

FastAPI launch cheyadaniki follow these terminal steps step by step:

- * **venv creation:** `python -m venv venv` (creates isolated local workspace for dependencies)
- * **venv activation:** `venv\Scripts\activate` (CMD console activates virtual python environment execution scope)
- * **install requirements:** `pip install -r requirements.txt` (installs FastAPI, SQLAlchemy, psycopg2 etc.)

5.3 Database Configuration - NullPool Pattern

WSL local db connection errors prevent cheydaniki connection pool patterns NullPool configure chesam. Idi static socket cache drops handle chesthundi. Connection code block look:

```
from sqlalchemy import create_engine
from sqlalchemy.pool import NullPool

# NullPool prevents Windows-WSL connection dropped cache errors
engine = create_engine(
    settings.DATABASE_URL,
    poolclass=NullPool
)
```

Chapter 6: Frontend (React.js Portal)

6.1 UI Design Guidelines

Web frontend responsive web saas dashboard guidelines implement chesam. No flashy icons or AI look gradients. Soft white cards with 12px border radius, clear typography layouts make it look like a medical dashboard portal.

6.2 React App Routing Layout

React Router App.jsx config routes define code mapping look:

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";

// ProtectedRoute logic restricts unauthorized access
const ProtectedRoute = ({ children, auth, allowedRoles }) => {
  if (!auth.token) return <Navigate to="/login" replace />;
  if (allowedRoles && !allowedRoles.includes(auth.role)) {
    return <Navigate to="/" replace />;
  }
  return children;
};
```

6.3 Dashboard Features Implementation

Admin Dashboard user table contains inline states sorting, filtering, and page offsets pagination configurations:

- * **Search Input:** Search field string values list items dynamic filter check query matching.
- * **Sorting logic:** Headers click events handle fields compare order (asc, desc) change state index.
- * **Pagination Toolbar:** Total records count divided in itemsPerPage offsets array pagination list pages.

Chapter 7: Authentication & JWT Flow

7.1 JWT Token Architecture

PillSync utilizes JSON Web Token (JWT) standard. JWT enables stateless security. Server signature verification check key variables check payload verify access control status.

```
[React App]                                     [FastAPI]
||                                               ||
|| ---- (POST /login with credentials) -----> ||
||                                               || (Hashed bcrypt
verify)
|| <--- (Return JWT token with role in payload) ----- ||
||                                               ||
|| ---- (HTTP requests with Header Authorization: Bearer JWT)-> ||
||                                               || (Token Signature
check)
|| <--- (Process payload and return protected data) ----- ||
```

7.2 Hashing Process & Security

Users password verify credentials check logic security.py helper bcrypt package base hashes data securely:

```
import bcrypt

def get_password_hash(password: str) -> str:
    # Generates a secure salt and hashes the plaintext password
    pwd_bytes = password.encode('utf-8')
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(pwd_bytes, salt).decode('utf-8')

def verify_password(plain_password: str, hashed_password: str) -> bool:
    # Compares the inputted password against database hash safely
    return bcrypt.checkpw(
        plain_password.encode('utf-8'),
        hashed_password.encode('utf-8')
    )
```

Chapter 8: API Endpoint Documentation

8.1 Auth endpoints mapping

- * **POST /api/auth/register/patient:** Registers a new patient account with a unique email.
- * **POST /api/auth/register/caregiver:** Registers a caregiver account and seeds caregiver table row profile.
- * **POST /api/auth/login:** Authenticates login password credentials and returns JWT token string.
- * **GET /api/auth/me:** Resolves user session identity profile status and returns user detail payload.

8.2 Admin dashboard metrics endpoints

- * **GET /api/admin/dashboard:** Returns counts for Total Users, Patients, Caregivers, Today's signups, active, inactive status.
- * **GET /api/admin/users:** Lists all user directory records (Full name, Email, Role, active status).
- * **GET /api/admin/patients:** Lists patient details database table profiles.
- * **GET /api/admin/caregivers:** Lists caregiver details database table profiles.

8.3 Sample API Response (GET /api/admin/dashboard)

```
{
  "total_users": 31,
  "total_patients": 20,
  "total_caregivers": 10,
  "todays_registrations": 1,
  "active_users": 31,
  "inactive_users": 0,
  "latest_users": [
    {
      "id": 1,
      "email": "admin@pillsync.com",
      "role": "admin",
      "created_at": "2026-07-05T14:41:09.949748"
    }
  ]
}
```

Chapter 9: Running the Project Step-by-Step

9.1 Step 1: Open VS Code & Terminals

First, VS Code opens codebase folders root context path `c:\Users\sukum\OneDrive\Desktop\PillSync`. Dhaantho files tree structure look visual avuthundhi.

9.2 Step 2: Start PostgreSQL Database

WSL console open chesi PostgreSQL service database checks run status start configure cheyyali. Command runs:

```
# Starts the postgresql cluster in WSL environment
sudo pg_ctlcluster 16 main start

# Check port status to confirm it's running
ss -tuln | grep 5432
```

9.3 Step 3: Launch FastAPI Backend Server

FastAPI boot up check main python context running. WSL virtual env configuration use chesi start commands run check:

```
cd backend
wsl ./venv_wsl/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

9.4 Step 4: Launch React Frontend Application

Windows CMD terminal open directory frontend path commands runs:

```
cd frontend
npm run dev
```

Expected Output: React client starts running on port 5173. Browser target `http://localhost:5173` direct login screens visual display load configure chesthundi.

Chapter 10: Debugging Common Errors

10.1 PostgreSQL Connection Refused (0x0000274D)

- * **Why it happens:** PostgreSQL database main service cluster WSL environment status down loop load context.
- * **How to solve:** WSL shell open `pg_ctlcluster` start commands running. WSL port config 5432 query check.

10.2 CORS Error (Cross-Origin Resource Sharing)

- * **Why it happens:** FastAPI server headers origins lists `http://localhost:5173` request verify block config.
- * **How to solve:** `main.py` `CORSMiddleware` configuration list check. Origin host matching verify updates check.

10.3 Port 8000 Already in Use

- * **Why it happens:** FastAPI uvicorn instance background task running. Port locks process blocks.
- * **How to solve:** Find PID holding port 8000 using `'netstat -ano | findstr 8000'` and kill process using `'taskkill /PID <PID> /F'`.

Chapter 11: Database Queries Analysis

11.1 Seeding query logic

Database seeding functions check sample metrics, users creation, profiles link, relations count logic calculations. Sample profiles users dynamic insertion loops in python ORM styles:

```
# Seeding logic loops mapping
user = User(
    email=email,
    hashed_password=hashed_pwd,
    role_id=patient_role.id,
    is_active=True
)
db.add(user)
db.flush() # dynamic returns user id

profile = PatientProfile(
    user_id=user.id,
    full_name=name,
    phone=phone,
    age=age,
    gender=gender,
    blood_group=blood,
    address=address,
    emergency_contact=emergency_contact
)
db.add(profile)
db.commit()
```

11.2 Stats Aggregations Queries

Today's registrations filter check query count operations check code. Combine date combine with timezone min check SQL counts:

```
# Query today's registrations since midnight today
today_start = datetime.combine(date.today(), time.min)
todays_registrations = (
    db.query(func.count(User.id))
    .filter(User.created_at >= today_start)
    .scalar()
)
```

Chapter 12: Mentor Explanation & Viva Prep

12.1 Project VIVA Vibe Check

Viva or mentor review panel presentation app features block, clear Semi-Telugu responses list standard review questions match:

* **Question 1:** Why are we using FastAPI instead of standard Django/Flask?

* **Answer 1:** FastAPI uses ASGI (Asynchronous Server Gateway Interface) so it is extremely fast. Django follows WSGI synchronous thread blocks. FastAPI Pydantic schema models automate request validation check chesthundi.

* **Question 2:** How does role validation secure routes in the system?

* **Answer 2:** API request authorization header bearer token pass chesthadhi. Backend Token decrypt chesi user role payload check query match matches allowed permissions. React routing system check role redirects check.

* **Question 3:** Where are the relation tables mappings defined?

* **Answer 3:** models/user_models.py file user relationships mappings ForeignKey reference keys config load define chesthayi.

Chapter 13: Milestone 1 Completion Summary

13.1 Milestone Scope Verification

Infosys Springboard Milestone 1 strict instructions checks completed. No excess medication tracking reminders templates code created yet to preserve scope bounds. Completed files mapping status check check:

- * **Security setup:** bcrypt hashing process, JWT encode decode, Role based endpoints validated.
- * **Database seeding:** 1 admin, 20 patients, 10 caregivers seeded successfully.
- * **Redirection UI logic:** Login redirects admin to admin portal dashboard, patients to patient pages, caregivers to caregiver roster directories.

Chapter 14: Project Demo Walkthrough

14.1 Click and Show Guide

Demo time sequence guidelines map dynamic portal views:

- * **Minute 0-2:** Landing page project layout explanation. Explain stack technologies utilized.
- * **Minute 2-4:** Register a patient, then login. Show patient personal profile edit manager. Save metrics database updates checks.
- * **Minute 4-7:** Login as Admin. Show the 6 statistics cards (Total, Patients, Caregivers, Today's, active, inactive). User Directory table search sort paginate demo.
- * **Minute 7-10:** Login as Caregiver. Show assigned Indian patients roster table details.

Chapter 15: Quick Revision Notes

15.1 Quick Reference Sheet

Review commands cheat sheet tables summary:

- * **Start Backend:** `wsl ./venv_wsl/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8000`
- * **Start Frontend:** `npm run dev` (in frontend folder CMD shell)
- * **Test Admin Account:** `admin@pillsync.com / password admin123`
- * **Test Patient Account:** `rahul.sharma@pillsync.com / password password123`
- * **Test Caregiver Account:** `ramesh.kumar@pillsync.com / password password123`

IMPORTANT NOTE: Check environment variables and NullPool configurations if local networking interfaces show timeout issues.